



UNIVERSIDAD DE MÁLAGA

Grado en Ingeniería Electrónica, Robótica y Mecatrónica

Memoria de Práctica

Lab-MR 4

Planificación de caminos I

Algoritmo de Dijkstra sobre grafo topológico

Ampliación de Robótica

Saúl Gutiérrez Rodríguez

Pablo Haro García

Mario Merino Prado

Repositorio: github.com/marioomerino24/ampliacion_robotica

Entorno: MATLAB

Índice

1. Objetivo	3
2. Implementación	3
3. Pruebas	4
3.1. Grafo de ejemplo (7 nodos)	4
3.2. Matriz J de grafos.mat (25 nodos)	5
4. Comentarios finales	6

1. Objetivo

Implementar en MATLAB una función con la signatura

```
[coste, ruta] = dijkstra(G, origen, destino)
```

que devuelva el coste y la secuencia de nodos del camino óptimo entre dos nodos de un grafo dirigido y ponderado, y validarla sobre los dos grafos del enunciado: el grafo de ejemplo de 7 nodos y la matriz J de grafos.mat (25 nodos, dirigida y asimétrica).

El convenio del enunciado es que $G(i, j) = 0$ representa ausencia de arco $i \rightarrow j$ y los pesos son no negativos, condición necesaria para que Dijkstra sea correcto.

2. Implementación

La función sigue el esquema clásico de Dijkstra (selección del nodo abierto con menor distancia, relajación de aristas, marcado como cerrado), por lo que aquí solo comentamos las decisiones que no son evidentes del enunciado.

Selección del mínimo por búsqueda lineal. Para los tamaños con los que se trabaja ($N \leq 25$), una búsqueda lineal sobre los nodos abiertos en cada iteración es más clara y rápida que mantener una cola de prioridad. La complejidad $O(N^2)$ resultante es perfectamente aceptable para este problema; con grafos mucho más grandes habría que reconsiderarlo.

Aceptar el struct de load directamente. El enunciado entrega los grafos dentro de grafos.mat. Para evitar el paso intermedio de extraer la matriz a mano, la función acepta tanto una matriz como el struct devuelto por load: si recibe el struct, busca el campo J (la matriz del enunciado) y, en su defecto, la primera matriz cuadrada numérica que encuentre. Así se puede invocar como:

```
[coste, ruta] = dijkstra(load('grafos.mat'), 1, 7)
```

sin pasos intermedios.

Corte temprano. Como interesa el camino a un destino concreto, el bucle se detiene en cuanto se extrae $u = t$ del conjunto abierto. No cambia la complejidad asintótica, pero reduce el trabajo medio.

Validaciones de entrada. Antes del bucle se comprueba que G es cuadrada, que los costes son finitos y no negativos (Dijkstra no soporta pesos negativos), y que los índices de origen y destino están dentro del rango. Si `origen == destino` se devuelve `coste = 0` sin entrar al bucle. Si la diagonal contiene autolazos, se avisa pero no se aborta: el filtro $G(u, v) > 0$ al recorrer vecinos hace que se ignoren automáticamente.

Reconstrucción de la ruta. Si $d[t] = \infty$ al terminar, se devuelve `coste = Inf` y `ruta = []` (destino inalcanzable). En caso contrario se reconstruye la ruta retrocediendo por el vector de predecesores hasta el origen.

3. Pruebas

3.1. Grafo de ejemplo (7 nodos)

Sobre el grafo G de 7 nodos del enunciado:

$$G = \begin{bmatrix} 0 & 2 & 0 & 0 & 9 & 0 & 0 \\ 2 & 0 & 1 & 2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 5 & 0 & 0 & 0 \\ 0 & 2 & 5 & 0 & 1 & 2 & 4 \\ 9 & 0 & 0 & 1 & 0 & 4 & 0 \\ 0 & 0 & 0 & 2 & 4 & 0 & 1 \\ 0 & 0 & 0 & 4 & 0 & 1 & 0 \end{bmatrix}$$

la llamada `[coste, ruta] = dijkstra(G, 1, 7)` devuelve `coste = 7` y `ruta = [1 2 4 6 7]`, que coincide con el resultado esperado del enunciado.

dijkstra(G, 1, 7) — coste = 7, ruta = [1 2 4 6 7]

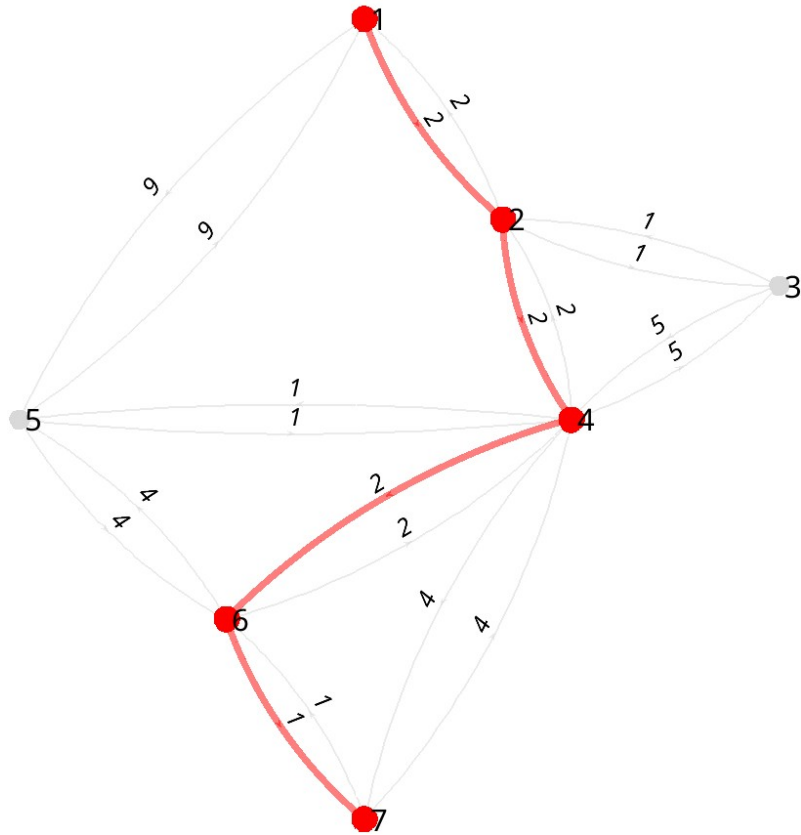


Figura 1: Grafo de 7 nodos con la ruta óptima de 1 a 7 resaltada en rojo. Coste 7, ruta [1 2 4 6 7].

3.2. Matriz J de grafos.mat (25 nodos)

La matriz J es un grafo dirigido y asimétrico de 25 nodos. La asimetría es relevante: por ejemplo $J(1,7) \neq J(7,1)$, lo que hace que las rutas de ida y vuelta entre el mismo par de nodos no sean iguales.

Tabla 1: Resultados de dijkstra sobre la matriz J (25 nodos, dirigida).

Origen	Destino	Coste	Ruta
1	19	16	[1 7 13 17 23 18 19]
19	1	19	[19 20 15 10 5 9 8 3 2 1]
25	19	11	[25 20 15 14 18 19]
19	25	4	[19 20 25]
1	23	12	[1 7 13 17 23]
23	1	22	[23 18 14 15 10 5 9 8 3 2 1]
1	24	∞	no alcanzable
23	22	40	[23 18 14 15 10 5 9 8 3 2 1 7 13 17 12 11 16 22]

Conviene fijarse en dos casos. Para $1 \rightarrow 23$ el coste es 12 y la ruta usa 5 nodos; para $23 \rightarrow 1$ el coste sube a 22 y la ruta a 10 nodos. Más extremo es $23 \rightarrow 22$: aunque 22 y 23 son nodos próximos en el mapa, los arcos disponibles obligan a recorrer casi todo el grafo (coste 40, 18 nodos). El caso $1 \rightarrow 24$ confirma que la función maneja correctamente la inalcanzabilidad: devuelve Inf y vector vacío sin lanzar error.

También se han probado los casos límite `origen == destino` (devuelve coste 0 y ruta de un solo nodo) y entrada con pesos negativos (rechazada con error en la validación).

4. Comentarios finales

La implementación cumple lo que pide el enunciado y se ha validado contra todos los casos propuestos. La búsqueda lineal del mínimo no es la opción más eficiente posible, pero para los tamaños de `grafos.mat` no merece la pena complicarla con una cola de prioridad; en grafos mucho más dispersos y grandes sería el siguiente paso a mirar.

Esta misma función se reutiliza en Lab-MR 5 como referencia para comparar contra A^* , y en Lab-MR 6 como planificador global cuya ruta se entrega luego a la navegación local reactiva.